BeStock

Documentation complète

Plateforme de gestion de stocks et d'inventaire

Générée le 22 août 2026 — version 1.0

Sommaire

1. Presentation
2. Architecture
3. Installation
4. Api
5. Modeles donnees
6. Securite
7. Frontend

1. Présentation

Qu'est-ce que BeStock ?

BeStock est une plateforme web de gestion de stocks et d'inventaire d'équipements. Elle permet de suivre le parc matériel d'une organisation : quels équipements existent, où ils se trouvent, dans quel état ils sont, et qui a fait quoi.

Fonctionnalités

Gestion du parc

- **Catalogue d'équipements** : création, modification, suppression avec référence unique, marque, modèle, numéro de série, date d'acquisition
- **Catégories** : classification (Informatique, Immobilier, Nettoyage...)
- **Statuts** : cycle de vie (Fonctionnel, Défectueux, En maintenance)
- **Emplacements** : localisation physique (bâtiment, étage, salle)

Suivi et traçabilité

- **Historique automatique** : chaque création / modification / suppression est journalisée avec l'auteur, les valeurs avant/après, la date et l'adresse IP
- **Tableau de bord** : statistiques en temps réel (total, en maintenance, derniers ajouts)
- **Recherche par catégorie** et pagination sur toutes les listes

Gestion des utilisateurs

- Inscription publique (compte `manager` inactif en attente de validation)
- Rôles hiérarchiques : `superadmin` > `admin` > `manager`
- Administration des comptes réservée au superadmin (activation, rôles, suppression)
- Authentification JWT avec access token + refresh token

Stack technique

Couche	Technologie	Version
Frontend	Nuxt (Vue 3, Pinia, Tailwind CSS)	4.x
Backend	FastAPI (Python)	0.141
ORM	SQLAlchemy 2 async	2.0
Base de données	PostgreSQL	16
Authentification	PyJWT (HS256) + bcrypt	—
Conteneurisation	Docker + Docker Compose	—

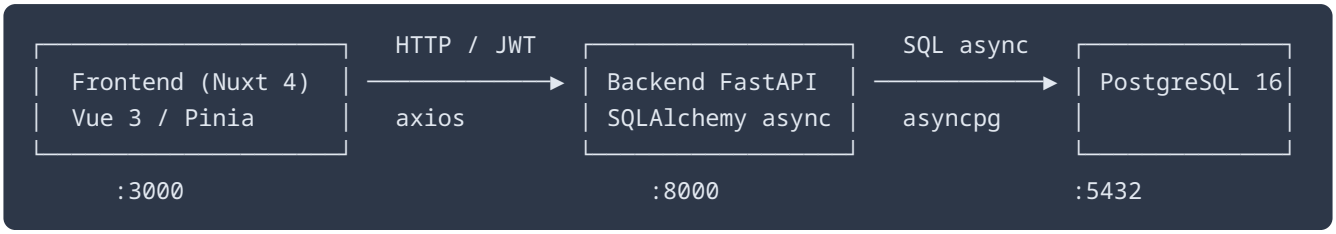
Captures des concepts clés



Chaque action métier passe par l'API authentifiée par JWT ; toute écriture sur un équipement génère une entrée dans l'historique consultable depuis l'interface.

2. Architecture

Vue d'ensemble



- **Communication frontend ↔ backend** : REST + JSON, tokens Bearer (access) et refresh token
- **Backend ↔ base** : driver `asyncpg`, sessions SQLAlchemy asynchrones
- **Conteneurs** : `bestock_db` (postgres:16-alpine), `bestock_api` (FastAPI + uvicorn)

Architecture en couches (backend)

Le backend suit une séparation stricte des responsabilités — chaque couche ne connaît que celle immédiatement en dessous :

```
api/routes/       HTTP : validation des entrées, codes de réponse, dépendances
    ↓
services/         logique métier : règles, garde-fous, journalisation historique
    ↓
crud/             accès aux données : requêtes SQLAlchemy réutilisables
    ↓
models/           schéma de la base (tables SQLAlchemy)
```

Responsabilités par dossier

Dossier	Rôle	Exemple
`app/api/routes/`	Endpoints REST, typage Pydantic	`equipements.py` expose GET/POST/PUT/DELETE
`app/api/deps.py`	Dépendances FastAPI : session DB, utilisateur courant, contrôle de rôles	`RolesDep(UserRole.SUPERADMIN)`
`app/services/`	Règles métier + journalisation	`create_equipement()` écrit aussi dans `historique`
`app/crud/`	Requêtes DB génériques	`list_equipements(db, skip, limit)`
`app/models/`	Tables SQLAlchemy	`User`, `Equipement`, `Historique` …
`app/schemas/`	Contrats d'entrée/sortie Pydantic	`EquipementCreate`, `UserRead`
`app/core/config.py`	Configuration via variables d'environnement (pydantic-settings)	`SECRET_KEY`, `DATABASE_URL`
`app/core/security.py`	Primitives crypto : bcrypt, JWT, refresh tokens	`hash_password()`, `decode_access_token()`
`app/db/`	Session async, création des tables, migrations légères, seed	`seed.py`
`app/middleware/`	Middlewares transverses	rate limiting, headers de sécurité

Architecture frontend

```
pages/          vues et navigation (file-based routing Nuxt)
    ↓
stores/         état global Pinia + actions API (auth, categories, equipements…)
    ↓
utils/api.js    client axios unique : base URL, injection du token, refresh auto
```

- `middleware/auth.global.js` : garde globale — redirige vers `/connexion` si non authentifié
- `plugins/auth.client.js` : hydrate le store auth depuis le stockage local au démarrage
- `layouts/dashboard.vue` : coquille de l'espace connecté (sidebar)

Cycle d'une requête authentifiée

1. Le client envoie `Authorization: Bearer <access_token>`
2. `deps.get_current_user` décode le JWT (`core/security.decode_access_token`)
3. L'utilisateur est chargé depuis la base ; s'il est inactif → **403**
4. Le contrôle de rôle éventuel (`require_roles`) peut renvoyer **403**

5. La couche service exécute la logique métier et journalise dans `historique`
6. La réponse est sérialisée via les schémas Pydantic (jamais d'objet ORM brut)

Arborescence complète

Voir README principal.

3. Installation

Prérequis

Outil	Version	Usage
Docker + Docker Compose	récent	backend + base de données
Node.js	≥ 20 (npm ou pnpm)	frontend
Python	≥ 3.12	uniquement pour du dev backend hors Docker

1. Développement — backend (API + PostgreSQL)

```
cd backend
docker compose up -d --build
```

Au démarrage, le conteneur exécute automatiquement :

1. `python -m app.internal_data` — création des tables + seed
2. `uvicorn app.main:app --host 0.0.0.0 --port 8000`

Service	URL
API	http://localhost:8000
Docs Swagger	http://localhost:8000/docs
Healthcheck	http://localhost:8000/health → `{"status": "ok"}`

Données initiales (seed)

- **Super admin** : créé depuis les variables `FIRST_SUPERUSER_EMAIL` / `FIRST_SUPERUSER_PASSWORD`
- **Catégories** : Informatique, Immobilier, Nettoyage
- **Statuts** : Fonctionnel, Défectueux, En maintenance
- **Emplacements** : Entrepôt, Bureau

Le seed est idempotent : il ne recrée rien si les données existent déjà.

Variables d'environnement

Copier le modèle puis adapter :

```
cp backend/.env.example backend/.env
```

Variable	Défaut	Description
`DATABASE_URL`	`postgresql+asyncpg://postgres:postgres@localhost:5432/bestock_db`	Connexion DB (`db` au lieu de `localhost` sous Docker)
`SECRET_KEY`	valeur d'exemple	Clé de signature JWT — **obligatoire en prod**, générer via `python -c "import secrets; print(secrets.token_urlsafe(32))"`
`ALGORITHM`	`HS256`	Algorithme JWT
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	Durée de vie access token
`REFRESH_TOKEN_EXPIRE_DAYS`	`7`	Durée de vie refresh token
`FIRST_SUPERUSER_EMAIL`	`admin@bestock.dev`	Email du compte initial
`FIRST_SUPERUSER_PASSWORD`	—	Mot de passe initial (**fort obligatoire**)
`FIRST_SUPERUSER_USERNAME`	`superadmin`	Nom d'utilisateur initial
`BACKEND_CORS_ORIGINS`	`["http://localhost:3000", "http://localhost:8080"]`	Liste blanche CORS

> **Jamais de mot de passe par défaut dans le code.** Le fichier `.env` est ignoré par git.

2. Développement — frontend

```
cd frontend        # ou npm run dev depuis la racine
npm install        # première fois seulement
npm run dev
```

Frontend sur **http://localhost:3000** — le client axios cible `http://localhost:8000/api/v1` (configurable dans `frontend/app/utils/api.js`).

3. Production

```
cp backend/.env.production backend/.env
# ÉDITER backend/.env :
#   - mots de passe forts (DB + super admin)
#   - SECRET_KEY fraîche
#   - domaines réels dans BACKEND_CORS_ORIGINS

docker compose -f docker-compose.prod.yml --env-file backend/.env up -d --build
```

Durcissements appliqués par `docker-compose.prod.yml` :

- Ports publiés sur `127.0.0.1` uniquement (reverse proxy Nginx/Traefik requis devant)
- Réseau Docker `internal` : la base n'est pas exposée publiquement
- Aucun secret en dur dans le compose — tout vient des variables d'environnement
- Uvicorn avec plusieurs workers

Commandes utiles

```
# Logs de l'API
docker logs -f bestock_api

# Redémarrer proprement toute la stack
docker compose restart

# Réinitialiser complètement la base (   destructif)
docker compose down -v && docker compose up -d --build

# Shell SQL dans la base
docker exec -it bestock_db psql -U postgres -d bestock_db
```

Résolution des problèmes

Symptôme	Cause probable	Solution
Port 8000 déjà utilisé	ancienne instance / autre service	`docker compose down` puis relancer
`address already in use` au build	conteneur orphelin	`docker compose up -d --build --force-recreate`
Login impossible après changement `.env`	super admin déjà seedé	modifier le mot de passe en base ou `down -v` pour re-seeder
401 systématiques	`SECRET_KEY` changée → tokens invalides	se reconnecter (les anciens JWT sont rejetés)

4. API REST

Généralités

- **Base URL** : `http://localhost:8000/api/v1`
- **Format** : JSON partout (sauf `DELETE` qui renvoie `204 No Content`)
- **Authentification** : header `Authorization: Bearer <access_token>`
- **Docs interactive** : http://localhost:8000/docs

Codes de retour standards

Code	Signification
200 / 201 / 204	Succès
401	Token absent, invalide ou expiré
403	Compte inactif, ou rôle insuffisant
404	Ressource introuvable
422	Payload invalide (validation Pydantic) — détail dans `detail`
429	Trop de requêtes (rate limiting)

Authentification — `/auth`

`POST /auth/register` → 201

Crée un compte (`manager`, inactif tant que validé par le super admin).

```
{ "email": "user@ex.fr", "password": "8caracteresmin", "username": "pseudo" }
```

Réponse : `UserRead`.

`POST /auth/login/json` → 200

```
{ "email": "admin@bestock.dev", "password": "••••" }
```

```
{
  "access_token": "eyJ...",
  "refresh_token": "chaîne-opaque-64-caractères",
  "token_type": "bearer"
}
```

> Rate limité à **5 req/min/IP**.

`POST /auth/refresh` → 200

Échange un refresh token valide contre un nouveau couple (rotation) :

```
{ "refresh_token": "..." }
```

→ même format que login. L'ancien refresh token est révoqué.

`POST /auth/logout` → 204

Révoque le refresh token fourni.

`GET /auth/me` → 200

Profil de l'utilisateur connecté (`UserRead`).

Utilisateurs — `/users` superadmin uniquement

Tous les endpoints exigent le rôle `SUPERADMIN` (sinon **403**).

Méthode	Route	Description
GET	`/users?skip=0&limit=50`	Liste paginée → `UserList {items, total, limit, skip}`
GET	`/users/{id}`	Détail
POST	`/users`	Création avec rôle imposé (`UserAdminCreate`)
PUT	`/users/{id}`	Modification : `username` , `password` , `is_active` , `role`
DELETE	`/users/{id}`	Suppression

`UserRead` :

```
{
  "id": 1, "email": "admin@bestock.dev", "username": "superadmin",
  "role": "superadmin", "is_active": true,
  "created_at": "2026-08-21T10:00:00Z", "updated_at": "..."
}
```

Référentiels — lecture seule

Méthode	Route	Retour
GET	`/categories`	`[{id, nom, description, actif}]`
GET	`/statuts`	`[{id, nom, description, actif}]`
GET	`/emplacements`	`[{id, nom, batiment, etage, salle, description, actif}]`

Accessibles à tout utilisateur authentifié et actif.

Équipements — `/equipements`

`GET /equipements` → 200

Paramètres : `skip` (défaut 0), `limit` (1–500, défaut 100), `categorie_id` (filtre optionnel).

```
{ "items": [EquipementRead], "total": 42, "limit": 100, "skip": 0 }
```

`GET /equipements/stats` → 200

Endpoint léger pour le dashboard :

```
{
  "total": 42,
  "maintenance": 3,
  "recent": [ /* 8 derniers équipements */ ]
}
```

> Paramètre optionnel `maintenance_statut_id` pour compter un autre statut.

`POST /equipements` → 201

```
{
  "reference": "EQ-2026-001",
  "nom": "Serveur Dell R740",
  "description": "texte libre",
  "marque": "Dell", "modele": "R740",
  "numero_serie": "SN-ABC123",
  "date_acquisition": "2026-01-15",
  "categorie_id": 1,
  "statut_id": 1,
  "emplacement_id": 2
}
```

Champs requis : `reference` (unique), `nom`, `categorie_id`, `statut_id`. → Journalise une entrée `historique` (action création).

Autres

Méthode	Route	Description
GET	`/equipements/{id}`	Détail (avec catégorie/statut/emplacement résolus)
GET	`/equipements/{id}/historique`	Historique complet de l'équipement
PUT	`/equipements/{id}`	Modification partielle → journalisée (avant/après)
DELETE	`/equipements/{id}`	Suppression → journalisée

Historique — `/historique`

`GET /historique?skip=0&limit=200` → 200

Journal global (limite max 1000). Entrée type :


```
{
  "id": 12,
  "utilisateur_nom": "superadmin",
  "equipement_id": 5,
  "action": "modification",
  "ancienne_valeur": "{\"statut\": \"Fonctionnel\"}",
  "nouvelle_valeur": "{\"statut\": \"En maintenance\"}",
  "date_action": "2026-08-21T14:32:00Z",
  "adresse_ip": "172.21.0.1"
}
```

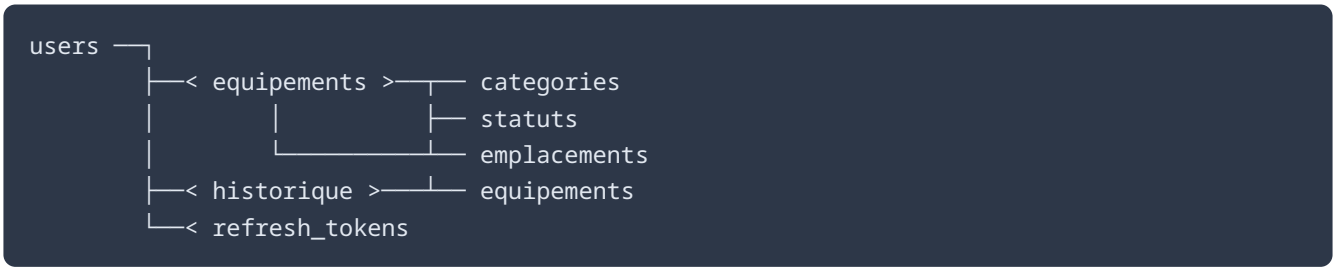
Santé

Méthode	Route	Description
GET	`/health` (hors `/api/v1`)	`{ "status": "ok" }` — pour supervision

5. Modèles de données

Vue d'ensemble

Base **PostgreSQL 16**, accès via **SQLAlchemy 2 async** (driver `asyncpg`).



Tables

`users`

Colonne	Type	Contraintes
id	INTEGER	PK, auto-increment
username	VARCHAR(50)	unique, non null
email	VARCHAR(100)	unique, non null
hashed_password	VARCHAR(255)	non null (bcrypt)
is_active	BOOLEAN	défaut `true`
role	ENUM `userrole`	`superadmin` / `admin` / `manager`, défaut `manager`
created_at / updated_at	TIMESTAMPTZ	auto

> Un compte nouvellement inscrit (`/auth/register`) naît avec `role=manager` et `is_active=false` : il ne peut pas se connecter tant que le super admin l'active.

`categories`

Colonne	Type	Contraintes
id	INTEGER	PK
nom	VARCHAR(100)	unique, non null
description	TEXT	nullable
actif	BOOLEAN	défaut `true`

Relation 1-N vers `equipements`.

`statuts`

Même structure que `categories` (nom en VARCHAR(50)). Valeurs seedées : **Fonctionnel**, **Défectueux**, **En maintenance**.

`emplacements`

Colonne	Type
id	INTEGER PK
nom	VARCHAR(100), non null
batiment	VARCHAR(100)
etage	VARCHAR(50)
salle	VARCHAR(100)
description	TEXT
actif	BOOLEAN

Permet une localisation fine : *Entrepôt / Bâtiment B / Étage 2 / Salle 204*.

`equipements` (table centrale)

Colonne	Type	Contraintes
id	INTEGER	PK
reference	VARCHAR(50)	**unique**, non null
nom	VARCHAR(150)	non null
description	TEXT	nullable
marque / modele	VARCHAR(100)	nullable
numero_serie	VARCHAR(150)	**unique**, nullable
date_acquisition	DATE	nullable
categorie_id	FK → categories.id	**non null**, indexé
statut_id	FK → statuts.id	**non null**, indexé
emplacement_id	FK → emplacements.id	nullable, indexé
created_by / updated_by	FK → users.id	nullable (traçabilité)
created_at / updated_at	TIMESTAMPTZ	auto

Relations ORM résolues dans les réponses API : `categorie`, `statut`, `emplacement`.

`historique` (journal d'audit)

Colonne	Type	Description
id	INTEGER	PK
utilisateur_id	FK → users.id	auteur de l'action (nullable)
equipement_id	FK → equipements.id	équipement concerné (nullable)
action	VARCHAR(50)	ex. `creation`, `modification`, `suppression`
ancienne_valeur	TEXT	JSON des valeurs avant modification
nouvelle_valeur	TEXT	JSON des valeurs après
date_action	TIMESTAMPTZ	horodatage
adresse_ip	VARCHAR(50)	IP cliente

La colonne calculée `utilisateur_nom` est renvoyée par l'API (jointure sur `users`, affiche `username` sinon email).

`refresh_tokens`

Colonne	Type	Description
id	INTEGER	PK
user_id	FK → users.id	propriétaire
token_hash	CHAR(64)	SHA-256 du token — **jamais le token en clair**
expire le champ d'expiration	TIMESTAMPTZ	contrôle à chaque usage

Un refresh token n'est vérifiable que par hachage comparé : une fuite de la base ne permet pas de reconstruire les tokens.

Migrations

Le schéma est créé et mis à jour au démarrage du conteneur par `app/db/init.py` (création des tables + migrations légères idempotentes, ex. renommage du type enum `userrole`). Pas d'outil Alembic : les évolutions sont gérées par ce script.

Seed (`app/db/seed.py`)

Idempotent, exécuté à chaque démarrage :

1. Super admin depuis `FIRST_SUPERUSER_*` (ignoré si l'email existe déjà)
2. Catégories : Informatique, Immobilier, Nettoyage
3. Statuts : Fonctionnel, Défectueux, En maintenance
4. Emplacements : Entrepôt, Bureau

6. Sécurité

Authentification

Mots de passe

- Hachage **bcrypt** avec sel aléatoire (`bcrypt.hashpw` + `gensalt`)
- Vérification en temps constant via `bcrypt.checkpw`
- Limite bcrypt de 72 octets gérée explicitement (troncature contrôlée)
- Minimum **8 caractères** imposé par les schémas Pydantic

JWT (access token)

Propriété	Valeur
Algorithme	HS256 (clé `SECRET_KEY`)
Durée de vie	30 min (`ACCESS_TOKEN_EXPIRE_MINUTES`)
Claims	`sub` (user id), `iat` , `exp` , `type=access`

Protections testées : - signature falsifiée rejetée (401) - attaque `alg:none` bloquée (liste blanche d'algorithmes dans `jwt.decode`) - token expiré rejeté

Refresh tokens (rotation)

- Génération cryptographique : `secrets.token_urlsafe(48)` (~64 caractères)
- Stockés **hachés SHA-256** — la base ne contient jamais le token en clair
- **Rotation à chaque usage** : `/auth/refresh` révoque l'ancien token et en émet un nouveau
- Révocation explicite via `/auth/logout`
- Durée : 7 jours

Autorisations (RBAC)

Hiérarchie des rôles :

```
superadmin  →  tout (gestion des utilisateurs incluse)
    admin   →  gestion du parc
     manager →  consultation + opérations courantes
```

Implémentation par dépendances FastAPI (`app/api/deps.py`) :

Dépendance	Effet si échec
`get_current_user`	401 (token invalide/expiré)
`get_current_active_user`	403 « Compte en attente de validation »
`require_roles(*roles)`	403 « Privilèges insuffisants »

Les routes `/users/*` exigent toutes `SUPERADMIN`.

Rate limiting

Middleware maison (**pur Starlette, zéro dépendance**) — `app/middleware/rate_limit.py` :

Route	Limite
`POST /auth/login/json`, `/auth/register`	5 req / minute / IP
`POST /auth/refresh`, `/auth/logout`	10 req / minute / IP

Dépassement → **HTTP 429** + header `Retry-After`. Fenêtre glissante en mémoire, clé = IP cliente.

Headers de sécurité

Ajoutés systématiquement (`app/middleware/security_headers.py`) :

Header	Valeur	Protection
`Content-Security-Policy`	`default-src 'self'; frame-ancestors 'none'; …`	XSS, clickjacking
`X-Frame-Options`	`DENY`	clickjacking
`X-Content-Type-Options`	`nosniff`	mime-sniffing
`Referrer-Policy`	`strict-origin-when-cross-origin`	fuite d'URL
`Permissions-Policy`	geolocation/micro/camera désactivés	API sensibles navigateur

> En production HTTPS, décommenter `Strict-Transport-Security` dans le middleware.

CORS

Liste blanche stricte configurée par environnement (`BACKEND_CORS_ORIGINS`) : - jamais `allow_origins=["*"]` combiné à `allow_credentials=True` - méthodes limitées aux verbes utilisés ; headers restreints (`Authorization`, `Content-Type`, `Accept`) - toute origine inconnue ne reçoit **aucun** header `Access-Control-Allow-Origin`

Vérifié par test : une requête préflight depuis `http://evil-attacker.com` est rejetée.

Validation des entrées (anti-injection)

Toutes les entrées passent par des schémas Pydantic **avant** tout accès base : - types et longueurs validés (ex. `reference ≤ 50`, `nom ≤ 150`) - email validé (`EmailStr`) - payload non JSON ou champ manquant → **422**, sans toucher la base

Requêtes 100 % paramétrées via SQLAlchemy (aucune concaténation SQL). Testé avec les payloads classiques (`' OR '1'='1`, `admin'--`, `DROP TABLE`) → 422/401, jamais 500.

Secrets et configuration

Règle	Application
Aucun secret dans le code	valeurs par défaut neutres uniquement, surchargées par `.env`
`.env` ignoré par git	présent dans `.gitignore`
`.env.example` public	modèle sans vraies valeurs
`SECRET_KEY` production	≥ 128 bits d'entropie, générée via `secrets.token_urlsafe(32)`
Comptes par défaut	aucun mot de passe par défaut embarqué ; super admin créé depuis l'environnement uniquement

Dépendances

Audités avec `pip-audit` (backend) et `npm audit` (frontend) : - backend : **0 vulnérabilité connue** - frontend : vulnérabilités critiques corrigées (Nuxt DevTools RCE, nanoid) ; seule reste une faille esbuild **Windows-only** (sans effet sous Linux)

Journalisation d'audit

Toute écriture sur un équipement produit une entrée `historique` : auteur, action, valeurs avant/après, horodatage, adresse IP. Consultable via `GET /api/v1/historique` et par équipement.

Limitations connues & pistes

- Rate limiting en mémoire : remettre à zéro au redémarrage ; pour du multi-instances, passer le compteur dans Redis
- Pas de verrouillage de compte après N échecs (le rate limiting limite déjà la force brute)
- HSTS à activer dès que TLS est en place

7. Frontend (Nuxt 4)

Stack

Élément	Technologie
Framework	Nuxt 4 (Vue 3, SSR)
État global	Pinia (`@pinia/nuxt`)
Styles	Tailwind CSS (`@nuxtjs/tailwindcss`)
Requêtes HTTP	axios (client unique `app/utils/api.js`)
Icônes	`@nuxt/icon` — collections heroicons + mdi **bundleées localement** (aucun appel réseau au runtime)
Typographie	`@nuxt/fonts`

Pages

Route	Fichier	Accès	Rôle
`/`	`pages/index.vue`	public	Landing page
`/connexion`	`pages/connexion.vue`	public	Login (email + mot de passe)
`/inscription`	`pages/inscription.vue`	public	Création de compte
`/dashboard`	`pages/dashboard.vue`		Statistiques (`GET /equipements/stats`), accès rapides
`/materiel/nouveau`	`pages/materiel/nouveau.vue`		Formulaire de création d'équipement
`/equipements/[id]`	`pages/equipements/[id].vue`		Fiche équipement + historique lié
`/categories`	`pages/categories/index.vue`		Liste des catégories
`/categories/[id]`	`pages/categories/[id].vue`		Équipements d'une catégorie
`/historique`	`pages/historique.vue`		Journal global (auteur, avant/après)
`/users`	`pages/users/index.vue`	superadmin	Gestion des comptes
`/compte`	`pages/compte.vue`		Profil personnel

> = protégé par le middleware global.

Middleware et plugins

`middleware/auth.global.js`

Garde exécutée sur **toutes** les navigations : - routes protégées + pas de session → redirection `/connexion` - évite la duplication de gardes dans chaque page

`plugins/auth.client.js`

Au chargement côté client : restaure la session depuis le stockage local puis tente un `refresh` silencieux si l'access token est expiré.

Stores Pinia

Chaque store suit le même modèle avec **cache mémoire** :

```
// pseudo-structure commune
state: { items: [], _loaded: false }
actions: {
  async fetch(force = false) {        // skippe l'appel si déjà chargé
    if (this._loaded && !force) return
    this.items = await api.get('...')
    this._loaded = true
  }
}
```

Store	Endpoint(s)	Particularité
`auth.js`	`/auth/login/json`, `/auth/me`, refresh/logout	conserve tokens, expose l'utilisateur courant
`categories.js`	`/categories`	cache long (référentiel stable), attribue les icônes (`pickIcon`)
`equipements.js`	`/equipements`, `/equipements/stats`	invalidation du dashboard après CUD
`historique.js`	`/historique`	pagination locale
`users.js`	`/users` (CRUD)	réservé superadmin

Invalidation automatique

Après toute création/modification/suppression, le cache concerné est marqué obsolète pour se recharger au prochain affichage (données toujours fraîches sans polling).

Client API — `utils/api.js`

Point central de toutes les requêtes :

- base URL : `http://localhost:8000/api/v1`
- injection automatique du header `Authorization: Bearer …`
- **intercepteur 401** : tente un `POST /auth/refresh`, rejoue la requête initiale ; si le refresh échoue → déconnexion et redirection `/connexion`

Layouts

- `layouts/default.vue` : navigation publique (landing, connexion…)
- `layouts/dashboard.vue` : coquille de l'espace connecté — sidebar, en-tête utilisateur

Icônes SVG

Configurées dans `nuxt.config.ts` :

```
icon: {
  serverBundle: { collections: ['heroicons', 'mdi'] },   // rendu SSR
  clientBundle: { scan: true }                           // détection auto + embarquement
}
```

Usage dans les composants : `<Icon name="heroicons:home" class="h-5 w-5" />`. Les icônes détectées sont embarquées dans le bundle → zéro appel réseau au runtime.

Lancer / builder

```
npm run dev        # développement (http://localhost:3000)
npm run build      # build production SSR
npm run generate   # pré-rendu statique
npm run preview    # sert le build
```